第 2 章 RKNPU 推理软件框架

2.1 推理软件框架

(1)RKNPU 硬件层

关于 RKNPU 硬件层在上个章节已经进行了讲解，这里就不再进行过多的赘述。

(2)RKNPU 驱动层

RKNPU 的驱动层是连接上层应用和 RKNPU 硬件的桥梁。驱动层的主要作用是将应用程序需要推理的内容提交给 RKNPU 进行计算，从而加速神经网络的训练和推理过程。具体来说，驱动层需要完成以下任务：

硬件初始化：驱动层需要初始化 RKNPU 硬件，包括设置寄存器、分配内存等操作，以确保 RKNPU 可以正常工作。

数据传输：驱动层需要将数据从主机内存传输到 RKNPU 内存中，以便进行计算。在计算完成后，驱动层还需要将计算结果传输回主机内存。

计算任务调度：驱动层需要根据应用程序的需求，管理和分配 RKNPU 的计算资源，以确保多个计算任务之间不会互相干扰。

(3)RKNPU 应用层

RKNPU 应用层由 AI 应用程序、RKNN API 以及运行时所需要的库所组成。

开发者通过调用瑞芯微提供好的 API 接口进行 AI 应用的开发，瑞芯微分别提供了 C 语言和 Python 语言这两种 API 帮助开发者进行嵌入式设备部署，Python 语言提供的接口较为简单，旨在帮助用户进行前期的模型检测、测试以及应用调试，而要想得到更好的效果从而真正应用

到实际项目中就要使用 C API 接口了。

而无论是由 C API 接口还是 Python 的 API 接口编写的应用程序，要想实际运行都需要相应的动态库，动态库包含这些 API 的具体实现，这些动态库由瑞芯微所提供。而我们只需要根据瑞芯微所提供的 API 编写对应的应用程序即可。

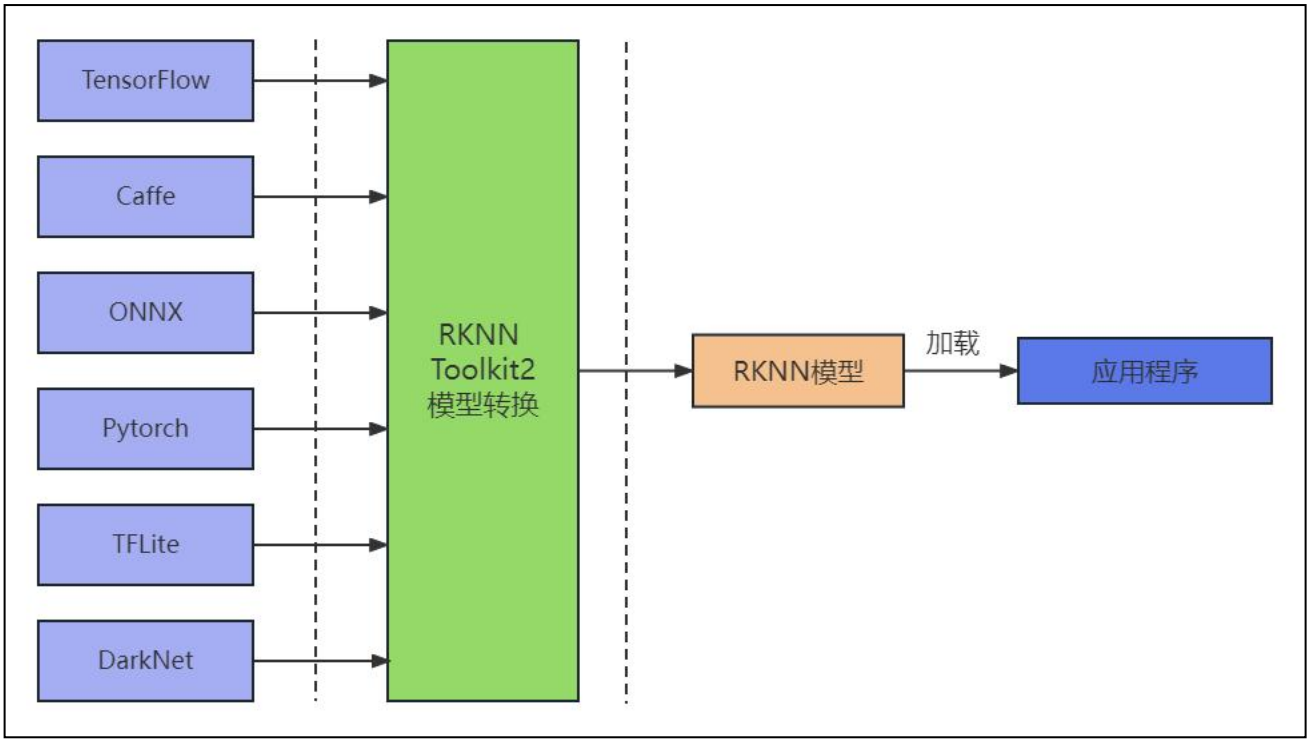
2.2 RKNN 模型

RKNN（Rockchip Neural Network）是瑞芯微公司开发的一种神经网络模型格式，它可以将常见的深度学习模型转换为适用于瑞芯微的 AI 加速器 RKNPU 的模型格式。RKNN 模型的优点是在保证精度的同时，可以实现高效的推理和低功耗的计算。下面是 RKNN 模型的详细介绍：

RKNN 模型的文件格式：RKNN 模型使用自定义的文件格式来表示神经网络模型。它将神经网络模型划分为两个部分：静态部分和动态部分。静态部分包括模型的网络结构和权重，而动态部分包括输入输出的形状和类型等信息。使用这种格式可以减少模型的存储空间和加载时间。

RKNN 模型的转换工具：为了将其他常见的深度学习模型转换为 RKNN 模型，瑞芯微提供了一个转换工具 RKNN-Toolkit2。该工具支持将 TensorFlow、Caffe、MXNet 等框架训练出来的模型转换为 RKNN 模型，并且支持对模型进行量化、融合等优化操作，以提高运行效率。

RKNN 模型的部署和推理：RKNN 模型可以通过 RKNPU 硬件进行高效的推理。在部署 RKNN 模型时，开发者需要使用 RKNPU 提供的 SDK，并调用相应的 API 接口来加载和运行 RKNN 模型。由于 RKNPU 硬件的优化，RKNN 模型可以实现高效的推理和低功耗的计算。



2.3 开源资料介绍

瑞芯微提供的 RKNPU 开发资料 github 链接为 https://github.com/rockchip-linux，本课程所使用的 SOC 为 RK3568 和 RK3588，归属于 RKNPU2，根据 RKNPU2 可以找到 RKNN Toolkit2 和 RKNPU2 SDK 两个开发相关资料，接下来将对上述两个开发工具包进行介绍。

2.3.1 RKNN Toolkit2

RKNN Toolkit2 仓库链接为 https://github.com/rockchip-linux/rknn-toolkit2。

RKNN-Toolkit2 资料包中有四个目录，分别为 docs、examples、packages 和 rknn_toolkit_lite2，

docs 目录包含了 RKNN-Toolkit2 的使用文档，包括更新记录、两个 Python 环境所需的依赖项、RKNN 算子支持列表、快速入门指南以及使用指导手册。

examples 目录包含了 RKNN-Toolkit2 的一些示例代码，用于演示 RKNN-Toolkit2 的一些功能。通过运行该示例代码，可以帮助用户更好的理解和掌握 RKNN-Toolkit2 的具体功能和使用方法。

packages 目录提供了 Python3.6 和 Python3.8 两个版本的安装包，分别对应 X86 平台的 ubuntu18 和 ubuntu20 两个发行版 Linux 操作系统。

而 rknn_toolkit_lite2 属于 RKNN-Toolkit2 的一个阉割版，会在下一小节进行讲解和对比。

下面对 RKNN-Toolkit2 进行简单的介绍：RKNN-Toolkit2 是瑞芯微为用户提供的、在 PC 端 ubuntu 平台进行模型转换、推理和性能评估的开发套件，用户通过该工具提供的 Python 接口可以便捷地完成以下功能：

- 模型转换：支持 Caffe、TensorFlow、TensorFlow Lite、ONNX、DarkNet、PyTorch 等模型转为 RKNN 模型，并支持 RKNN 模型导入导出，RKNN 模型能够在 Rockchip NPU 平台上加载使用
- 量化功能：支持将浮点模型量化为定点模型，目前支持的量化方法为非对称量化（asymmetric_quantized-8 及 asymmetric_quantized-16），并支持混合量化功能。asymmetric_quantized-16 目前版本暂不支持
- 模型推理：能够在 PC 上模拟 NPU 运行 RKNN 模型并获取推理结果或将 RKNN 模型分发到 Rockchip NPU 设备上进行推理并获取推理结果
- 性能评估和内存评估：将 RKNN 模型分发到 Rockchip NPU 设备上运行，以评估模型在实际设备上运行时的性能和内存使用信息
- 量化精度分析：该功能将给出模型量化前后每一层推理结果与浮点模型推理结果的余弦距离，以便于分析量化误差是如何出现的，为提高量化模型的精度提供思路
- 模型加密功能：使用指定的加密等级将 RKNN 模型整体加密。因为 RKNN 模型的加密是在 NPU 驱动中完成的，使用加密模型时，与普通 RKNN 模型一样加载即可，NPU 驱会自动对其进行解密。

至于 RKNN-Toolkit2 的环境搭建以及具体使用会在之后的章节中进行讲解。

2.3.2 RKNN Toolkit lite2

RKNN Toolkit lite2 相较于 RKNN-Toolkit2 多了一个 lite 后缀，可以简单的理解为 RKNN-Toolkit2 的阉割版，只保留了推理的功能,可以帮助用户在开发板端进行模型的初步部署和测试。

RKNN-Toolkit lite2 资料包中有三个目录，分别为 docs、examples 和 packages。

docs 目录包含了 RKNN-Toolkit lite2 的使用文档。

examples 目录包含了 RKNN-Toolkit lite2 的示例代码，由于 RKNN-Toolkit2 lite2 的使用较为简单所以只提供了一个示例。

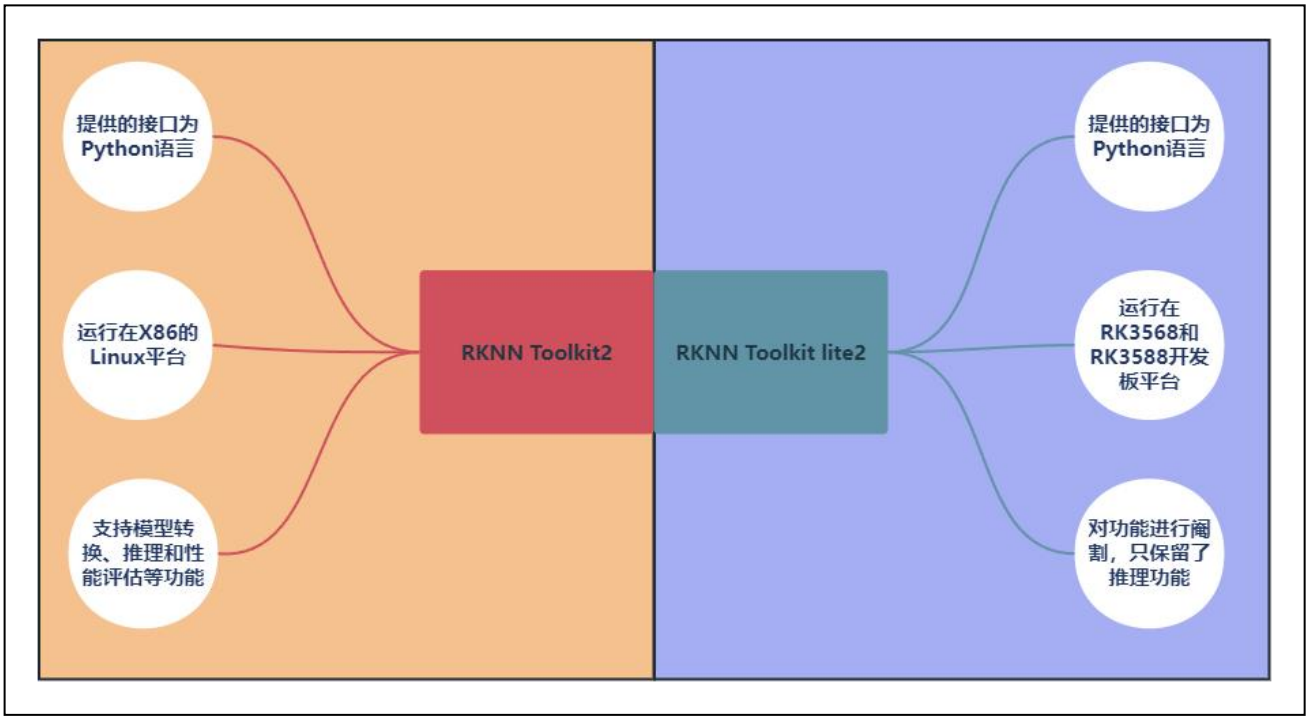
packages 目录提供了 Python3.7 和 Python3.9 两个版本的安装包，需要注意的是这里的安装包架构为 aarch64，即需要安装在 RK3568 或者 RK3588 的 ubuntu 等发行版 Linux 操作系统上。

最后对 RKNN-Toolkit2 和 RKNN-Toolkit lite2 进行对比，对比如下：

（1）提供的语言接口都是 Python 语言。

（2）RKNN-Toolkit2 提供的安装包架构为 X86_64，运行在日常使用的 PC 电脑上，一般情况下安装在虚拟机 ubuntu 上，而 RKNN-Toolkit lite2 提供的安装包架构为 aarch64，运行在 RK3568 或者 RK3588 开发板上。

（3）RKNN-Toolkit2 可以进行模型转换、推理和性能评估，而 RKNN-Toolkit lite2 只保留了推理功能，用于模型的初步部署和调试。



2.3.3 RKNPU2 SDK

RKNPU2 SDK 的仓库链接为 https://github.com/rockchip-linux/rknpu2。

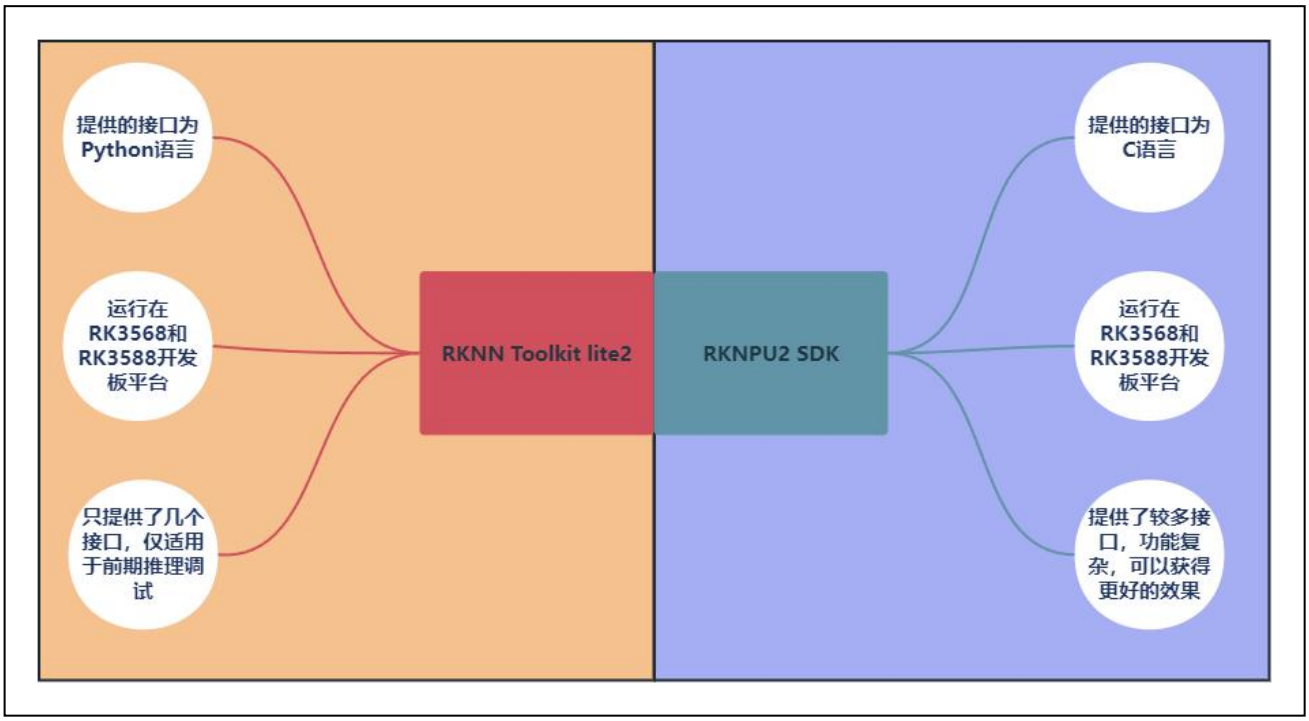
RKNPU2 SDK 资料包中有 3 个目录，分别为 docs、examples 和 runtime。

docs 目录包含了 RKNPU2 SDK 的使用文档，包括 RKNN 算子支持列表、快速入门指南以及使用指导手册。

examples 目录包含了 RKNPU2 SDK 的一些示例代码和第三方库。通过这些示例代码，可以帮助我们更快速的掌握 RKNPU2 所提供的一些 C API 接口和使用流程。

runtime 目录用于存放 API 运行时依赖的动态库和头文件，并且提供了一个名为 rknn_server 的可执行程序，在后面 RKNN-Toolkit2 连板推理时会用到。

RKNPU2 SDK 和 RKNN-Toolkit lite2 要实现的功能相同，都是在开发板部署 RKNN 模型，然后进行推理，只是相较于 RKNN-Toolkit lite2 提供了更多的接口，调用起来较为复杂，但可以获得更好的运行效果。

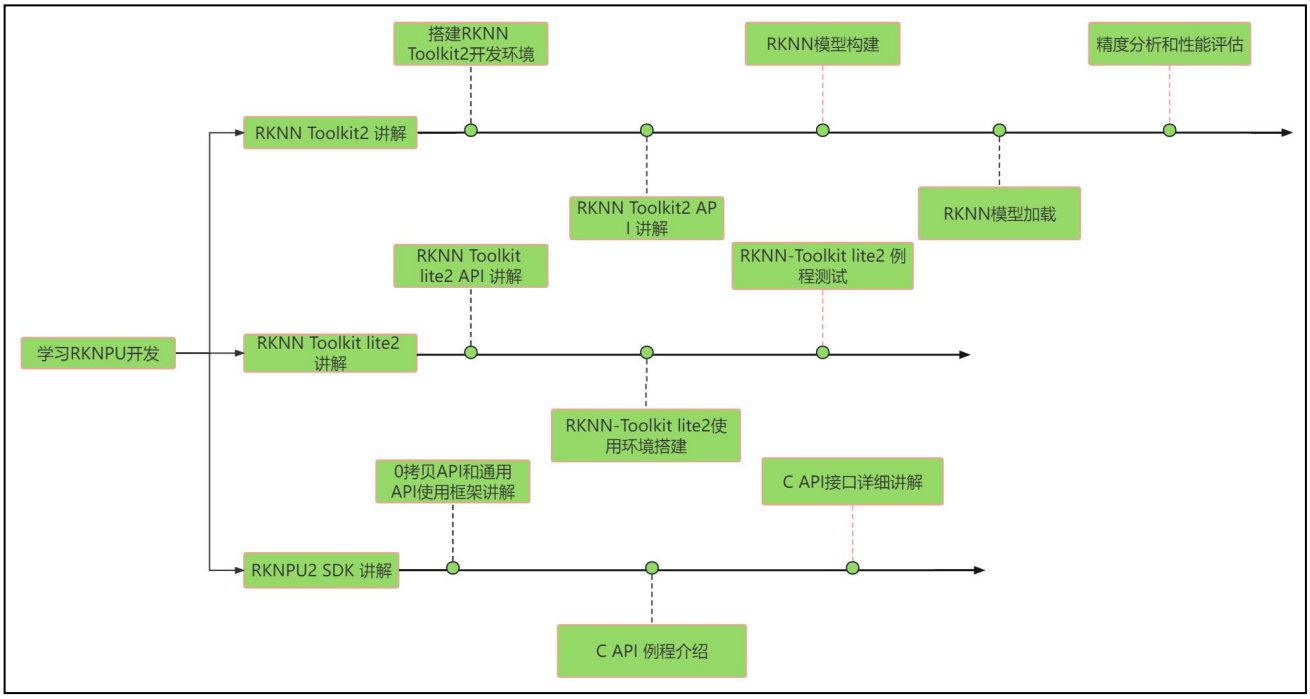


2.4 学习步骤整理

无论使用 RKNPU2 SDK 提供的 C API 接口进行 RKNN 模型部署，还是使用 RKNN-Toolkit lite2 提供的 Python API 接口进行 RKNN 模型部署，在此之前，都需要先使用 RKNN Toolkit2 将网络模型转换为 RKNN 模型，或者对模型进行精度分析等操作，所以在 RKNPU2 开发阶段，首先要学习 RKNN-Toolkit 2 的使用。

RKNN-Toolkit 2 的学习分为以下几个步骤，搭建 RKNN-Toolkit 2 开发环境、讲解 RKNN-Toolkit 2 提供的 API 接口并测试常用功能、对 RKNN 模型构建、精度分析、性能评估、内存评估以及

RKNN 模型加载等重要接口进行单独讲解。

在 RKNPU 开发过程中，RKNN-Toolkit 2 就是用来构建并且评测 RKNN 模型的，得到符合我们大致预期的 RKNN 模型之后就可以进行项目的部署了，模型部署分为 RKNN-Toolkit lite2 部署和 RKNPU2 SDK 部署，由于 RKNN-Toolkit lite2 提供的接口与 RKNN-Toolkit 2 提供接口相似，且数量较少，主要用于初期调试，所以我们首先会讲解 RKNN-Toolkit lite2 部署，包括 RKNN-Toolkit lite2API 接口讲解、RKNN-Toolkit lite2 使用环境搭建、RKNN-Toolkit lite2 例程测试。

RKNN-Toolkit lite2 讲解完成之后就到了最难的 RKNPU2 SDK 提供的 C API 接口讲解了，C API 根据处理数据时的方法可以分为 0 拷贝 API 和通用 API，首先会根据例程讲解两种 API 的使用框架、随后对瑞芯微提供的 C API 例程进行介绍，凭借这些例程可以帮助我们后期进行快速开发，在例程的讲解中会对出现的各个 C API 进行详细的讲解。